

**UNIVERSIDADE DE BRASÍLIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: SEGURANÇA COMPUTACIONAL**

PROF. ROBERTO RODRIGUES FILHO

**RELATÓRIO TÉCNICO:
IMPLEMENTAÇÃO DO WHATSCHAT
SEGURO**

TRABALHO II – TRABALHO FINAL

1 Descrição Completa da Solução Desenvolvida

A aplicação **WhatsChat** foi projetada sob o paradigma de arquitetura distribuída cliente-servidor utilizando a linguagem Python. O sistema divide-se em duas entidades principais: o `server.py`, que atua como um nó central de encaminhamento e gestão de presença, e o `client.py`, que encapsula a interface do utilizador e a totalidade das primitivas criptográficas de aplicação.

1.1 Fluxo de Comunicação e Topologia da Rede

O servidor central não possui visibilidade sobre o conteúdo confidencial das mensagens trocadas, atuando estritamente como um intermediário (*router*) de pacotes estruturados em formato JSON. A comunicação estabelece-se sobre sockets TCP estruturados da seguinte forma:

1. **Fase de Conexão (mTLS):** O cliente estabelece uma ligação TCP com o servidor. Sob a camada de transporte, ocorre o *handshake* TLS mútuo, onde ambas as partes provam a sua identidade através de certificados digitais X.509 validados por uma Autoridade de Certificação (CA) partilhada.
2. **Fase de Descoberta (/list):** O utilizador pode solicitar ao servidor a listagem em tempo real de todas as identidades (membros) autenticadas e ativas no sistema.
3. **Fase de Acordo de Chaves (/connect):** Para iniciar uma conversa com um par, o cliente dispara um protocolo de acordo de chaves Diffie-Hellman efêmero através do servidor. O servidor repassa os parâmetros públicos sem intervir no cálculo.
4. **Fase de Chat Seguro:** Com o canal estabelecido, as mensagens passam a ser assinadas individualmente com códigos de autenticação de mensagem baseados em hash (HMAC) antes do envio.

2 Mecanismo Adotado para Autenticação

A autenticação no WhatsChat afasta-se do modelo vulnerável baseado em credenciais estáticas (utilizador e palavra-passe) e adota o **mTLS (Mutual Transport Layer Security)**. Este mecanismo garante que o processo de verificação de identidade abranja não apenas a máquina ou o processo em execução, mas valide de forma inequívoca o próprio utilizador detentor da chave privada.

2.1 Infraestrutura de Chaves Públicas (PKI) Local

Utilizando o ecossistema OpenSSL, foi criada uma Autoridade de Certificação Raiz própria (`ca.crt`). Esta CA é a raiz de confiança do sistema. O servidor e cada utilizador individual (ex: Alice e Bob) possuem o seu próprio par de chaves assimétricas (RSA de 2048 ou 4096 bits) e um certificado X.509 assinado digitalmente pela CA.

2.2 O Processo de Validação Bidirecional

No código do servidor, o contexto SSL é configurado rigorosamente com a flag `ssl.CERT_REQUIRED`:

```
1 context.verify_mode = ssl.CERT_REQUIRED
2 context.load_verify_locations(cafile="ca.crt")
```

Durante o *handshake* de transporte:

1. O servidor apresenta o seu certificado (`server.crt`) ao cliente, que verifica a assinatura digital contra a `ca.crt`.
2. O servidor exige o certificado do cliente. O cliente envia o seu certificado assinado (ex: `alice.crt`) e prova a posse da chave privada correspondente através de uma assinatura gerada em tempo de execução.
3. O servidor valida a cadeia do certificado do cliente. Se for legítimo, o servidor utiliza o método `conn.getpeercert()` para extrair dinamicamente o campo `commonName` (CN) do assunto do certificado.

Este CN extraído é utilizado como a identidade oficial do utilizador dentro da aplicação. Não há possibilidade de falsificação de identidade (*spoofing*), pois qualquer tentativa de injetar um nome diferente do contido no certificado assinado resultará na rejeição imediata da conexão pela camada de segurança do Python. Caso um atacante tente ligar-se sem um certificado válido assinado pela CA, o alerta de falha de autenticidade é disparado e a ligação é abortada.

3 Mecanismo Adotado para Garantir Confidencialidade

A confidencialidade dos dados trafegados no WhatsChat é assegurada de ponta a ponta na camada de transporte criptográfica gerida pelo protocolo TLS (versões 1.2 ou 1.3), implementado nativamente através do módulo `ssl` do Python.

3.1 Cifragem de Fluxo e Proteção contra Intercepção

Ao envolver o socket TCP bruto através da função `context.wrap_socket()`, todo o fluxo de dados subsequente deixa de trafegar em texto claro e passa a ser cifrado de forma simétrica (tipicamente utilizando cifras AES-GCM ou ChaCha20-Poly1305, dependendo da negociação automatizada do TLS).

A eficácia deste mecanismo pode ser verificada experimentalmente através da captura de pacotes de rede com a ferramenta **Wireshark**. Ao analisar o tráfego gerado pela aplicação, constata-se o seguinte comportamento:

- Os pacotes iniciais expõem o *Handshake* do TLS (troca de cifras suportadas e envio de certificados públicos).
- A partir do momento em que o canal seguro é estabelecido, todos os pacotes subsequentes contendo payloads de chat (comandos como `/list`, strings JSON contendo mensagens e hashes) passam a ser catalogados no Wireshark estritamente sob o rótulo **"Application Data"**.

- O conteúdo interno dos pacotes exibe apenas sequências de bytes pseudoaleatórias de alta entropia. Mesmo que um atacante execute um ataque de interceptação ativa na rede local, a leitura ou extração do conteúdo das conversas torna-se computacionalmente inviável sem acesso às chaves simétricas de sessão geradas no *handshake*.

4 Mecanismo Adotado para Garantir Integridade

A integridade e o não-repúdio das mensagens ao nível da aplicação são garantidos através do acoplamento de dois protocolos criptográficos fundamentais: o **Acordo de Chaves de Diffie-Hellman** e o **HMAC (Hash-based Message Authentication Code)**.

4.1 Estabelecimento do Segredo via Diffie-Hellman

Para gerar a chave necessária para o HMAC sem que ela trafegue pela rede (o que violaria a segurança), os clientes utilizam o Diffie-Hellman. Dado um número primo P e um gerador G públicos:

1. O Utilizador A gera uma chave privada secreta X_A e calcula a sua chave pública $Y_A = G^{X_A} \pmod{P}$.
2. O Utilizador B gera uma chave privada secreta X_B e calcula a sua chave pública $Y_B = G^{X_B} \pmod{P}$.
3. Eles trocam as chaves públicas através do comando `/connect`.
4. O Utilizador A calcula o segredo: $K = (Y_B)^{X_A} \pmod{P}$.
5. O Utilizador B calcula o mesmo segredo: $K = (Y_A)^{X_B} \pmod{P}$.

4.2 Geração e Verificação do HMAC

Uma vez calculado o segredo partilhado K , as mensagens trocadas passam a utilizar os módulos `hmac` e `hashlib` do Python.

Antes de enviar a mensagem, o remetente calcula o código de integridade:

$$\text{HMAC} = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel \text{Mensagem})) \quad (1)$$

Utilizando a função SHA-256, o hash gerado é anexado ao payload JSON enviado. Ao receber o pacote, o destinatário reconstrói o HMAC esperado utilizando o segredo K guardado em memória e a mensagem recebida. A validação é feita através de uma comparação estrita de tempo constante:

```
1 hmac.compare_digest(expected_mac, received_mac)
```

Se um atacante tentar injetar bits ou modificar caracteres da mensagem durante o tráfego, o cálculo do HMAC no destino resultará num valor completamente diferente. A aplicação deteta imediatamente a divergência e exibe o alerta: **[ALERTA DE INTEGRIDADE]** A mensagem foi adulterada durante a transmissão!, descartando o texto corrompido.

5 Dificuldades Encontradas e Aprendizados Obtidos

5.1 Principais Dificuldades

1. **Gestão da Infraestrutura de Certificados (PKI):** A configuração correta das restrições de assinatura no OpenSSL e a geração manual de extensões para assegurar a autenticação mútua (mTLS) exigiram uma curva de aprendizagem acentuada, especialmente no ajuste dos campos de *Common Name* (CN) e validação de caminhos locais das chaves.
2. **Sincronismo Criptográfico Fim-a-Fim:** Coordenar o estado da troca de chaves Diffie-Hellman assíncrona através de threads em segundo plano (`threading.Thread`) revelou-se complexo. Garantir que uma mensagem de chat não fosse processada antes da conclusão do cálculo do segredo compartilhado exigiu uma estruturação rigorosa de buffers lógicos no cliente.

5.2 Aprendizados Obtidos

A execução prática do WhatsChat solidificou conceitos teóricos complexos de Segurança Computacional. O grupo compreendeu a necessidade prática da defesa em profundidade: enquanto o mTLS protege o canal contra espreitadores na rede (Confidencialidade e Autenticação de transporte), o Diffie-Hellman combinado com o HMAC protege a aplicação contra servidores maliciosos ou comprometidos que tentem alterar dados no meio do caminho (Integridade fim-a-fim). A experiência provou que a segurança de um sistema distribuído depende diretamente do isolamento correto de responsabilidades de cada camada criptográfica.